can run on the edge network, which are geographically located servers. This means the security checks are performed closer to the user to improve the overall performance and security of your web application.

Listing 9.2 shows a typical middleware pipeline that encapsulates some of the aforementioned security controls and filters. This code is a reference implementation of a composable middleware function you can use.

Listing 9.2 Composable middleware

```
import { NextResponse } from 'next/server';
const composeMiddleware = (middlewares) => {
  return async (request) => {
    let response = NextResponse.next();

    for (const middleware of middlewares) {
      try {
        const result =
await middleware(request, response);
        if (result.response) return result.response;
        if (result.continue === false) break;
      } catch (error) {
        console.error('Middleware error:', error);
        return NextResponse.json(
          { error: 'Internal Server Error' },
          { status: 500 }
        );
      }
    }

    return response;
  };
};
const middlewareChain = composeMiddleware([
  handleCORS,
  rateLimit,
  authenticate,
  securityHeaders,
]);

export async function middleware(request) {
  return await middlewareChain(request);
}

export const config = {
  matcher: '/api/:path*',
};
```

Takes an array of middleware functions as input and returns a single, composed middleware function

Executes middleware and handles results

Composes all middleware functions

The main middleware function that Next.js will execute for matching routes

Defines the paths that the middleware should be applied to

Here the `composeMiddleware` function takes an array of individual middleware functions (`handleCORS`, `rateLimit`, etc.) and creates a single, composed middleware function. This composed function then iterates through each middleware in the provided array, executing them sequentially. Each middleware can modify the response, short-circuit the